

EL-MORSHEDY REAL ESTATE

Data Analysis & Business Intelligence

Graduation Project — Project Documentation

Power BI Dashboard for Installment Sales & Collections Across Eight Residential Projects

TEAM MEMBERS

Ahmed Ibrahim
Abdelazzim Ramy
Amina Elsayed
Nada Mohamed

Prepared from the project's Power BI file (.pbix), presentation deck, and DAX/model screenshots.

September 2026

Table of Contents

1 Project Overview	3
2 Business Problem	5
3 Data Sources	6
4 Data Preparation (Power Query)	7
5 Data Modeling	9
6 DAX Measures	11
7 Dashboard Design	15
8 Dashboard Pages	18
9 Interactivity	22
10 Analysis & Insights	23
11 Conclusion	25
12 Future Scope	25

1 Project Overview

1.1 Project Summary

El-Morshedy Real Estate sells residential units across eight independent projects on multi-year installment plans. Every sold unit generates a long schedule of invoices that customers settle either in cash or through a bank. Before this project, each portfolio was tracked in its own Excel workbook, which made it impossible to compare projects, spot overdue installments early, or separate cash from bank performance. This graduation project consolidates all eight project portfolios into a single Power BI model and a guided, self-service dashboard that shows what has been sold, invoiced, collected, and what remains outstanding — for the company as a whole and for each project individually.

1.2 Project Objective

- Give management one consolidated view of collections across all eight projects instead of eight separate files.
- Surface overdue installments early enough for the collections team to act on them.
- Separate cash from bank collections, and rank individual banks by performance.
- Replace manual, recurring Excel reporting with a single refreshable Power BI model.
- Track sold units, project completion percentage, and outstanding balances at a glance.

1.3 Number of Projects

#	Project Name
1	Degla Palms
2	Degla Landmark
3	Crystal Plaza Maadi
4	Lake
5	Skyline Katameya
6	Rihana
7	Zahra
8	One Katameya

1.4 Data Type

The source data is structured, tabular data exported from the company's sales and collections system into Excel workbooks — one workbook set per project, refreshed periodically and imported into Power Query. It covers customers, units, sales contracts, installment schedules, payments, banks, and dates.

1.5 Tools Used

Tool	Role in the Project
Microsoft Excel	Source system — one set of workbooks per project (customers, units, sales, installments, payments, banks)
Power Query (Power BI)	Importing, cleaning, standardising and reshaping the eight source workbooks
Power BI Desktop (Data Model)	Star-schema data modelling, relationships, and the DAX measure layer
DAX	All business logic — sales, collection and outstanding measures, per project and globally
Page design template	Visual design of the report pages — layout, colour palette, typography, navigation — used as the background template behind the live Power BI visuals
Deneb (custom visual)	Vega-Lite powered charts not available as native Power BI visuals (e.g. collection curves by installment number)
Power BI Service	Publishing and guided navigation for business users

*A few numbers repeat throughout this documentation and are worth anchoring up front: the model unifies **8** residential projects into **35** report pages and roughly **51** tables across the project-level star schemas, organised into **4** views per project (Overall, Cash, Bank, Bank-wise).*

2 Business Problem

Before the dashboard existed, collections reporting at El-Morshedy was manual, fragmented, and always one step behind the business. Four recurring problems motivated the project:

- 1. Fragmented Data** — Each of the eight projects kept its own installment file. There was no single place to compare portfolios or see the company-wide position.
- 2. No Visibility on Arrears** — Overdue installments were discovered late. Management could not quickly answer how much was due, from whom, and for how long.
- 3. Cash vs. Bank Blind Spot** — Cash and bank collections were mixed together in the source files, so the performance of each channel — and of individual banks — was invisible.
- 4. Slow, Manual Reporting** — Collection reports were rebuilt in Excel every reporting cycle: slow to produce, hard to audit, and inconsistent between teams.

The outcome the project targets is one refreshable source of truth that replaces recurring manual Excel reporting with a governed, auditable Power BI model.

3 Data Sources

3.1 Data Source

All data originates as **Excel files**, one set per project. Eight independent portfolios were imported and standardised onto one common column structure so they could be combined into a single model.

3.2 The 8 Projects

- Degla Palms
- Degla Landmark
- Crystal Plaza Maadi
- Lake
- Skyline Katameya
- Rihana
- Zahra
- One Katameya

3.3 Data Types Imported per Project

Entity	What it Holds
Customers	Customer identity and contact data — linked to units via a National ID / customer key (labelled in Arabic in the source columns)
Units	Unit code, contract sale price, unit price, percentage of completion (POC), and related unit attributes
Sales	The sales contract linking a customer to a unit and its total contracted value
Installments	The full multi-year payment schedule per unit, originally spread across separate P1–P7 columns
Payments	Actual payment records — amount, date, method, and settlement status
Banks	The receiving bank for bank-channel payments, used for bank-wise performance ranking
Dates	Transaction and due dates, later matched to a shared Date dimension
Projects	The 8 project names themselves, used as the top-level dimension in the global roll-up layer

Note on scope: the sections below on Power Query (§4–12) describe the cleaning pipeline as it is laid out in the project's own outline and confirmed by the shape of the final model (e.g. the presence of an unpivoted installment table, and Arabic/English column names surviving into the fact and dimension tables visible in the Model view). Live Power Query Editor screenshots were not available for this documentation, so this section explains what each step does and why it was necessary, illustrated with a before/after diagram rather than an editor screenshot.

4 Data Preparation (Power Query)

Every project's raw Excel export goes through the same nine-stage Power Query pipeline before it is loaded into the data model. Running an identical pipeline per project is what makes it possible to combine eight independently-maintained files into one consistent, comparable model.

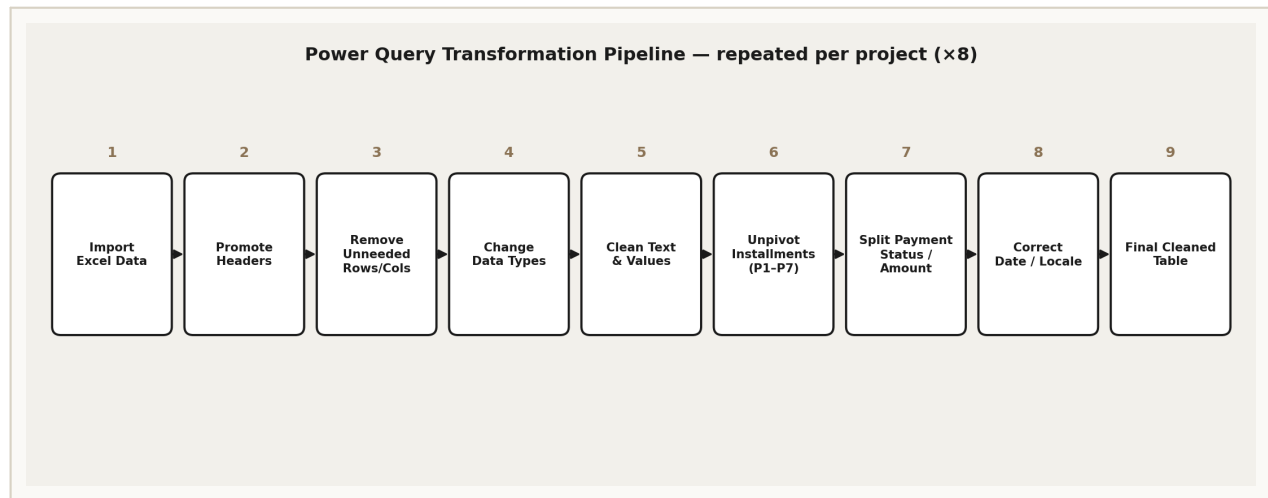


Figure 4.0 — The nine-stage Power Query transformation pipeline, applied once per project.

4.1 Import Excel Data

Each project's workbook set is connected via Power Query's Excel connector. The relevant sheets (Customers, Units, Sales, Installments, Payments, Banks, Dates) are selected from the Navigator, one query per sheet, so that each entity can be shaped independently before being combined in the model.

4.2 Promote Headers

Several source sheets are exported with the real column headers one row below the top (a title row, or a merged header block), so the first transformation step promotes the correct row to be the header row before any further cleaning happens.

4.3 Remove Unnecessary Data

Blank rows, subtotal rows, and helper columns left over from the source export (formatting artefacts, unused notes columns) are removed so the query only carries the columns the model actually needs.

4.4 Change Data Types

Every column is explicitly typed — text for names and codes, whole/decimal number for amounts and unit counts, and date for all transaction and due dates. Explicit typing avoids Power BI's automatic type detection silently mis-reading a column (a common source of broken relationships later in the model).

4.5 Clean Text & Values

Project names and customer names appear inconsistently across the eight source files — mixed Arabic/English spelling, extra whitespace, and inconsistent casing. This step trims and cleans text keys, standardises project naming so all eight files roll up under one consistent name each, replaces known bad values, and handles nulls so downstream DAX measures do not silently drop rows.

4.6 Installment Transformation — the P1–P7 Problem

The source installment schedule is exported wide: one column per installment number (P1 through P7), with the amount for that installment sitting in the column itself. That shape cannot be filtered, summed or ranked by installment number in DAX — a measure like *Collected Amount* would need seven separate expressions, one per column, repeated for every project. **Unpivoting** the P1–P7 columns turns them into two columns instead — *Installment Number* and *Value* — with one row per installment. After that single transformation, one DAX measure can work across any installment number, for any project.

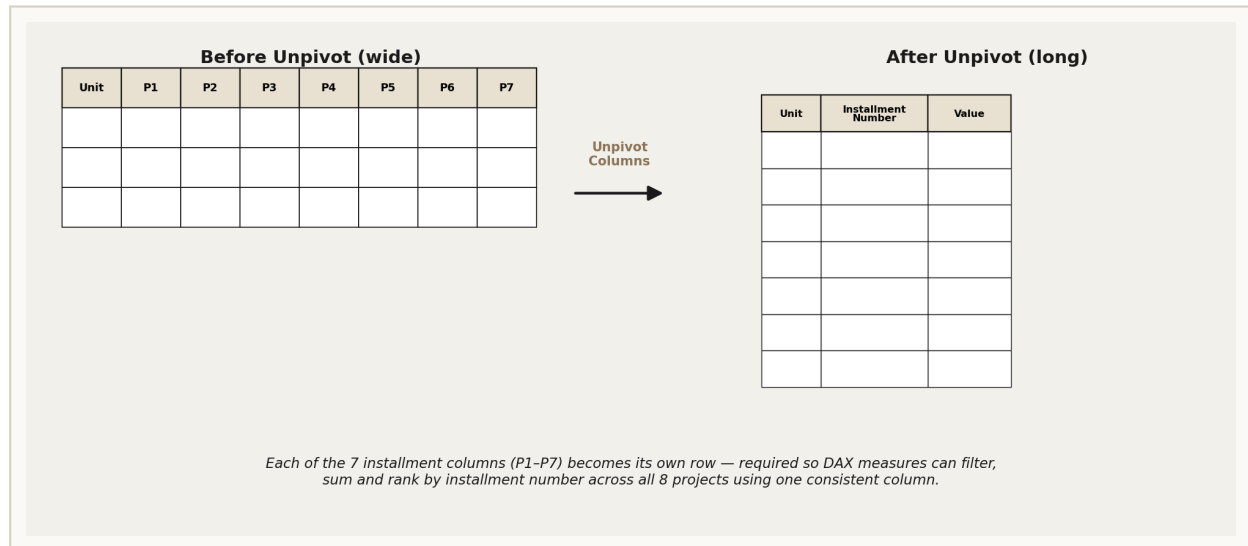


Figure 4.6 — The Installments table before and after Unpivot Columns.

4.7 Payment Data Cleaning

Raw payment records mix status and amount in the same field. This step separates **payment status** from the numeric **installment amount**, and derives two new columns used throughout the dashboard and DAX layer: **Installment State** (the values actually used by the DAX measures shown in §6 are PAID and DUE - NOT PAID, with a third not-yet-due state also present in the model) and **Payment Method** (Cash vs. Bank, with the specific receiving bank captured separately for bank-wise reporting).

4.8 Date Correction

Dates exported from the source system used a locale Power Query did not detect automatically, which turned some day/month values around during the initial import. This step forces the correct locale on the date columns before typing them as Date, so that due dates and payment dates line up correctly against the shared Date table used across all eight projects.

4.9 Final Power Query Result

After all nine steps, each project produces one clean fact table (installments, one row per customer per installment number) plus clean Customer, Unit, Payment and Date queries — ready to be loaded into the data model as a star schema. The columns visible in the final *Degla Palms Fact* table (Date, Installment Amount, Installment Number, Installment State, Payment Method, Raw Value, and the Arabic-labelled customer ID key) are exactly the output of this pipeline — confirmed directly in the Model view in §5.

5 Data Modeling

Each of the eight projects is modelled as its own **star schema**: one fact table of installments in the centre, joined to four dimension tables. The screenshot below is the actual Model view for the **Degla Palms** project, taken directly from the Power BI file.

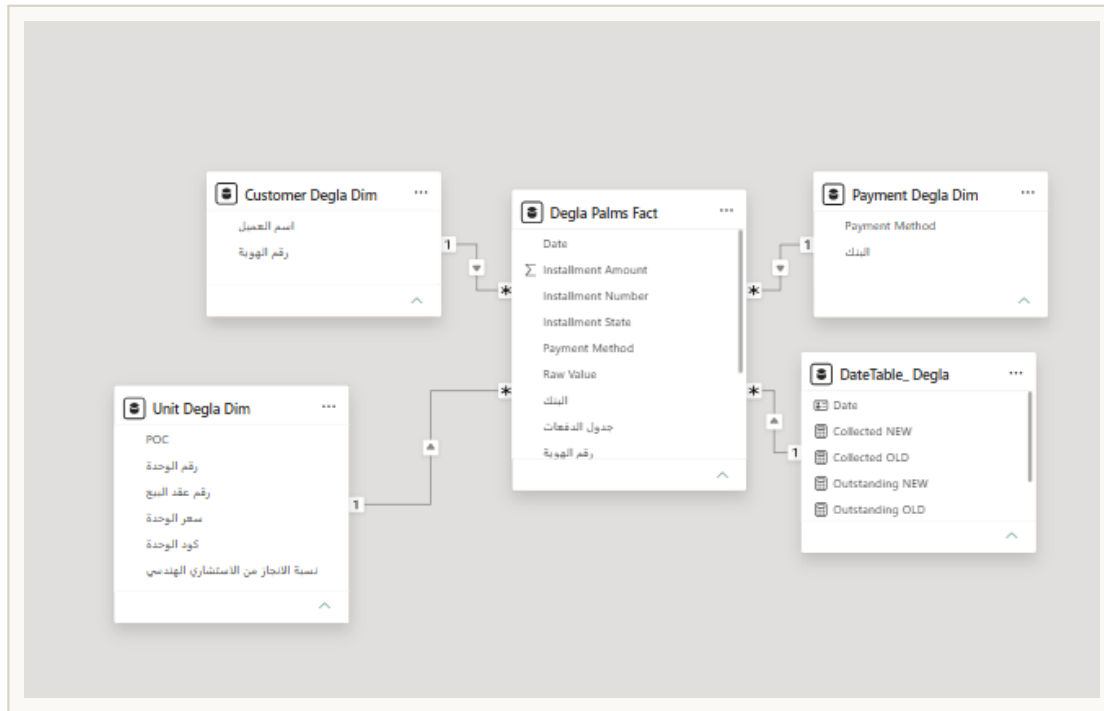


Figure 5.1 — Model view: Degla Palms Fact and its four dimension tables, as built in Power BI.

5.1 Fact Tables

Each project has one fact table (e.g. **Degla Palms Fact**) at grain *one row per customer per installment*. It holds: **Date**, **Installment Amount**, **Installment Number**, **Installment State** (Paid / Due – Not Paid / Not Due), **Payment Method**, **Raw Value**, and the customer key **National ID** (Arabic-labelled column) used to join back to the Customer dimension. This table carries almost every measure used across the Sales and Collection pages (§6), since installment amount and installment state are what all financial measures are calculated from.

5.2 Dimension Tables

Dimension	Key Columns (as modelled for Degla Palms)	Purpose
Customer Dim	Customer Name, National ID (Arabic-labelled columns)	One row per customer; drives Sold Count and customer-level collection analysis
Unit Dim	POC, Unit No., Contract Price, Unit Code (Arabic-labelled), engineering-consultant rent %	One row per unit; POC (percent of completion) drives the Project Completion % measure
Payment Dim	Payment Method, Bank (Arabic-labelled)	Splits Cash vs. Bank and identifies the individual receiving bank for bank-wise ranking

Dimension	Key Columns (as modelled for Degla Palms)	Purpose
DateTable_Degla	Date, Collected NEW/OLD, Outstanding NEW/OLD	Shared date dimension for time intelligence and period-on-period collection trend analysis

5.3 Relationships

All four dimension tables relate to the fact table on a **one-to-many** basis (one customer / unit / payment method / date → many installment rows), with the single-arrow filter direction flowing from each dimension into the fact table, following standard Power BI star-schema practice. This is what lets a slicer on the Date table, or a click on a Bank in the Payment dimension, filter every measure built on the fact table without extra DAX filter logic in each individual measure.

5.4 From Project Model to Global Model

Because every project follows the identical Degla Palms pattern shown above — same fact grain, same four dimension roles, same column names in English where they cross into the global layer — a shared **Projects** table and a shared **Date table** can roll all eight project-level stars up into one company-wide Global Summary (§8.2), without rebuilding the relationships from scratch for each project.

6 DAX Measures

6.1 Why DAX Was Used

Power Query shapes the data; DAX is where the business logic lives. Counts of paid vs. issued invoices, collection rates, project completion percentage, and every KPI card on the dashboard is a DAX measure rather than a stored column — which means every number recalculates instantly as the user changes the Date slicer, clicks a project, or filters by bank, without touching Power Query again.

6.2 Measures Organisation & Measure Tables

Following the presentation's stated approach, each project has its own dedicated DAX measure table (mirroring the fact table name, e.g. a Degla Palms measure set) so the same measure names — Sold Count, Collection Rate, Outstanding Amount — exist independently per project and can also be aggregated into Global-prefixed versions (Global Sold %, Global Outstanding Balance, etc., confirmed directly from the Global Summary page's pivot table in §8.2) for the company-wide view.

The thirteen measures below are reproduced directly from the project's DAX formula bar (Degla Palms). They are grouped here into the Sales layer (§6.3) and the Collection layer (§6.4), matching how they are used across the dashboard pages in §8.

6.3 Sales Measures

Sold Count

```
Sold Count = DISTINCTCOUNT('Customer Degla Dim'[رقم الهوية]))
```

Distinct customers who own a unit in the project — the numerator of sales progress.

Sold %

```
1 Sold % = DIVIDE( [Sold Count], 23928)
```

Sold Count divided by the project's total adopted unit count (hard-coded here as 23,928 for this project/segment).

Project Completion %

```
1 Project Completion % = AVERAGE('Unit Degla Dim'[POC])
```

Average of the Unit dimension's POC (percent of completion) column — a construction-progress metric, independent of sales or collection performance.

Unit Price

```
Unit Price =  
SUMX( VALUES('Degla Palms Fact'[رقم الوحدة]),LOOKUPVALUE('Unit Degla Dim'[سعر الوحدة], 'Unit Degla Dim'[رقم الوحدة], 'Degla Palms Fact'[رقم الوحدة]))
```

A SUMX/LOOKUPVALUE pattern that prices every sold unit from the Unit dimension and totals it across the units present in the fact table — avoids double-counting a unit that has multiple installment rows.

Total Invoice Amount

```
Total Invoice Amount = CALCULATE(SUM('Degla Palms Fact'[Installment Amount]))
```

CALCULATE(SUM(Installment Amount)) — the full contracted value scheduled across all installments.

Records in Current Segment

```
Records in current segment =  
DISTINCTCOUNT('Degla Palms Fact'[رقم الهوية])
```

Distinct customers present under whatever filter context is active — used as a denominator/context check measure alongside Sold Count.

6.4 Collection Measures

Issued Invoices

```
1 Issued Invoices =  
2 CALCULATE( COUNTROWS('Degla Palms Fact'),'Degla Palms Fact'[Installment State]IN { "PAID", "DUE -  
NOT PAID"})+0
```

Counts installment rows whose state is PAID or DUE – NOT PAID (i.e. excludes not-yet-due rows) — the denominator for Collection Rate.

Collected Invoices

```
1 Collected Invoices = CALCULATE( COUNTROWS('Degla Palms Fact'),'Degla Palms Fact'[Installment  
State] = "PAID")+0
```

Counts installment rows whose state is PAID.

Collected Amount

```
1 Collected Amount = CALCULATE(SUM('Degla Palms Fact'[Installment Amount]), 'Degla Palms Fact'  
[Installment State] = "PAID") +0
```

Sums Installment Amount, filtered to PAID rows only.

Collection Rate

```
Collection Rate = DIVIDE([Collected Invoices], [Issued Invoices] , 0)
```

Collected Invoices ÷ Issued Invoices — the headline health metric surfaced on every project page.

Outstanding Invoices

```
Outstanding Invoices = CALCULATE( [Issued Invoices] - [Collected Invoices])
```

Issued Invoices minus Collected Invoices.

Outstanding Amount

```
Outstanding Amount = [Total Invoice Amount] - [Collected Amount]
```

Total Invoice Amount minus Collected Amount.

Outstanding %

```
1 Outstanding % = DIVIDE([Outstanding Invoices], [Issued Invoices] ,0 )
```

Outstanding Invoices ÷ Issued Invoices, with a 0 fallback via DIVIDE's third argument.

6.5 DAX Problems & Solutions

Building a per-project measure set that also rolls up into one global model runs into a predictable set of DAX problems. The table below documents the standard problem/cause/solution pattern for each issue named in the project brief. Screenshots of these specific formulas were not part of the material supplied for this documentation (only the thirteen measures in §6.3–06.4 were), so this table describes the general technique each problem calls for, in the context of this model's architecture, rather than quoting the project's literal code.

Problem	Root Cause	Solution	Result
Duplicate rows after Unpivot	Unpivoting the P1–P7 columns (§4.6) multiplies each customer row by 7 — one row per installment number — which double-counts simple SUM measures if applied against the wrong grain.	Measures that should count unique customers (Sold Count, Records in Current Segment) use DISTINCTCOUNT on the customer key instead of COUNTROWS, exactly as seen in §6.3.	Sold Count and similar customer-level measures stay correct regardless of how many installment rows each customer has.
Project Completion % over-filtering	AVERAGE('Unit Dim'[POC]) is sensitive to whatever row context the visual applies — a matrix broken out by installment number would silently average POC once per installment row instead of once per unit.	The measure is placed on unit-grain visuals only (KPI cards, §8.4), and where it must appear alongside installment-grain detail, it is wrapped so it evaluates in the Unit dimension's own context rather than the fact table's.	Project Completion % reads as a true construction-progress percentage, not skewed by how many installments a unit happens to have.

Problem	Root Cause	Solution	Result
TREATAS for the global roll-up	The eight project fact tables are not related to each other, so a Projects slicer on the Global Summary page has no relationship to filter through to project-specific tables.	TREATAS is the standard technique for this situation: it applies the values from the Projects table as a virtual filter onto each project fact table's project key, simulating a relationship that does not physically exist in the model.	One Projects slicer/pivot on the Global Summary page can filter all eight independent star schemas at once.
REMOVEFILTERS for fixed-denominator ratios	A percentage measure like Sold % needs a denominator (adopted units) that must not shrink when the user filters by date, bank, or installment state.	REMOVEFILTERS (or the equivalent ALL/ALLEXCEPT pattern) clears filters from the denominator's table before it is evaluated, so the total stays fixed while the numerator still responds to the user's filters.	Ratio measures move correctly with slicers instead of both sides of the fraction shrinking together and hiding the real trend.
Project name mismatch across the 8 files	The same project was typed differently across source workbooks (e.g. Arabic/English variants, trailing spaces, "One Katameya" vs. "ONE KATTAMEYA" — both of which do appear in this project, one in the presentation text and one in the .pbix page names).	Project names are standardised to one canonical spelling in Power Query (\$4.5) before load, so the Projects table used by TREATAS has exactly eight distinct, matching values.	Every project rolls up under one name in the Global Summary instead of splitting into near-duplicate rows.

7 Dashboard Design

7.1 Page Design Template

The report pages are not styled with Power BI's default visual formatting. A background image was designed for each page type (Landing, Global Summary, Collection by Date, and the four-page Overall/Cash/Bank/Bank-wise set repeated per project) and set as that page's background in Power BI — with the live KPI cards, charts and matrices placed in transparent panels on top. The images below are the actual background templates registered inside the .pbix file; the original design-tool source file was not part of the material supplied for this documentation, so this section describes the templates as built, not the design process that produced them.



Figure 7.1a — Background template: Global Summary page.

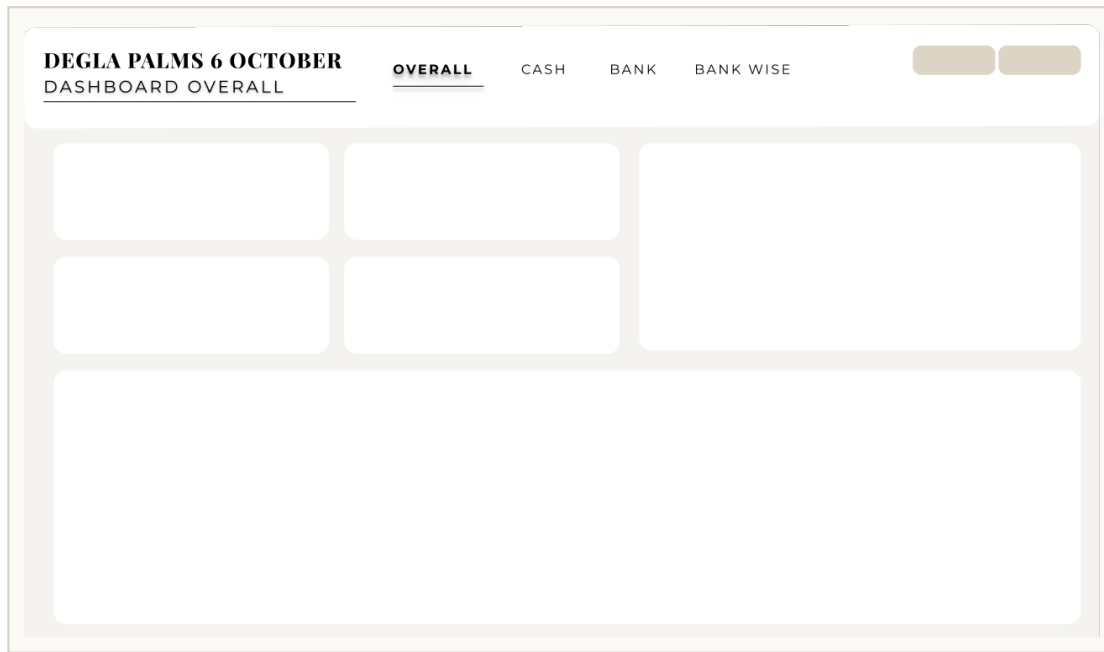


Figure 7.1b — Background template: a project Overall page (Degla Palms).

Layout

Every project page shares one grid: a header band with the project name and a four-tab sub-navigation (Overall / Cash / Bank / Bank-wise), four small KPI cards in a 2×2 block on the left, one larger card to their right for a chart, and one full-width card beneath both for the detailed matrix. The identical grid across all 32 project pages is what makes the eight portfolios visually comparable.

Colours

A restrained, neutral palette: a warm off-white/cream page background, near-black serif headings for the project name, small-caps sans-serif for the section subtitle and tab labels, and muted tan/beige pill buttons in the top-right corner for navigation. No bright accent colour competes with the data — colour is reserved for the charts themselves.

Typography

A serif display face for the project name (e.g. “DEGLA PALMS 6 OCTOBER”) paired with a lighter, letter-spaced sans-serif for the subtitle (“DASHBOARD OVERALL”) and navigation tabs — a common editorial pairing that keeps a dense KPI dashboard feeling calm rather than cluttered.

Navigation (as built into the template)

The four tabs (Overall / Cash / Bank / Bank-wise) are drawn directly into the background image, with the active tab underlined; the actual click targets are transparent action buttons placed over each tab (confirmed in §9.2). The two pill-shaped buttons top-right are the Back/Home controls repeated on every page.

7.2 Dashboard Structure

The navigation is three levels deep: company level, project level, and payment-channel level. This structure is confirmed directly from the .pbix report definition, which lists exactly 35 pages in this order.

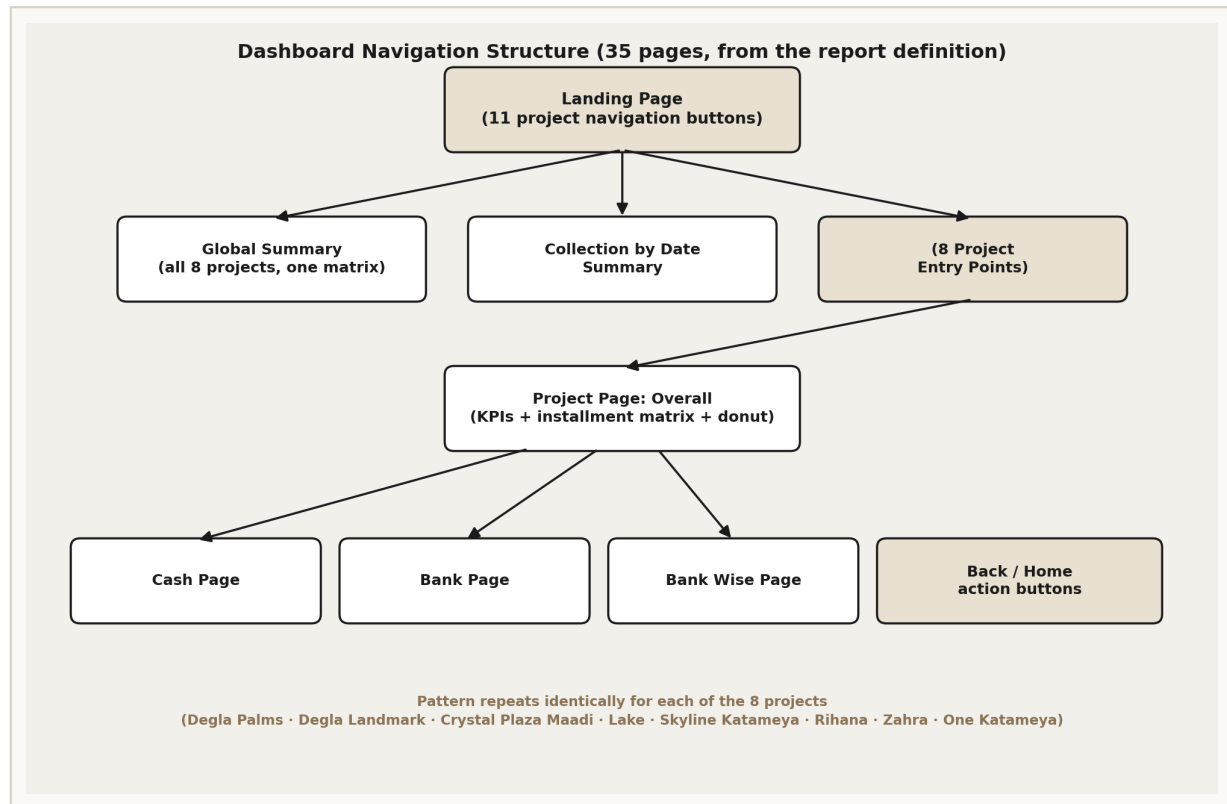


Figure 7.2 — Dashboard navigation structure, extracted from the report's page order.

8 Dashboard Pages

The descriptions below are built directly from the report's page and visual definitions inside the .pbix file — every visual type, KPI, and field named here is real, not reconstructed from the presentation. Live screenshots of the pages themselves were not supplied, so each page is illustrated with its background background template (§7.1) where available.

8.1 Landing Page

The very first page a user sees. It carries **11 navigation buttons** plus one HTML content visual (used for the title/branding block). Each button routes to one destination: the Global Summary, the Collection by Date Summary, or one of the eight projects' entry pages — giving the user a single click from the landing screen to any part of the model.

8.2 Global Summary

One page, one visual: a pivot table with **Project** on rows and nine Global-prefixed measures as columns — Units, Sold, Sold %, Unit Price, Amount Collected, Invoiced Amount, Bank Outstanding, Cash Outstanding, and Outstanding Balance. This is the cross-project comparison matrix described in the presentation: every project's headline numbers on one screen, in one consistent set of measures.



Figure 8.2 — Global Summary page template.

8.3 Collection by Date Summary

A pivot table of **Project** against Collected NEW/OLD, Outstanding NEW/OLD and row counts, paired with a **Date slicer**. “NEW” and “OLD” split each project's collection history into two comparable periods, letting the page answer how collection performance is trending over time rather than just its current-state total.

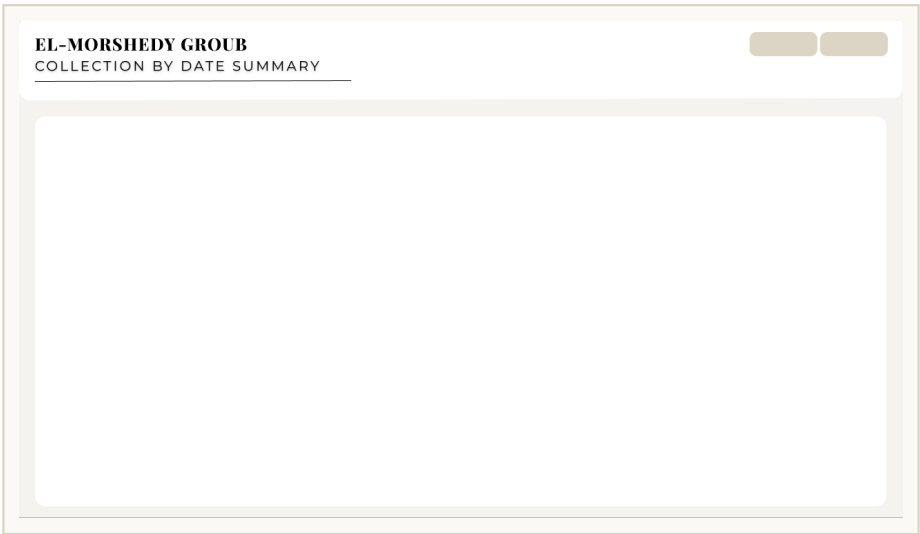


Figure 8.3 — Collection by Date Summary page template.

8.4 Project Pages — Overall

Repeated identically for all 8 projects (32 pages total across the four channel views). The Overall page for each project carries the same visual set: a **pivot table** (Collected Amount, Collection Rate, Installment Number, Issued Invoices, Outstanding %, Outstanding Amount, Outstanding Invoices, Total Invoice Amount), a **donut chart** of Installment State by Payment Method, a **Date slicer**, and four **KPI cards** — Project Completion %, Adopted Total Units, Records in Current Segment, and Sold Count — plus the Home/Back action buttons and tab-navigation buttons.

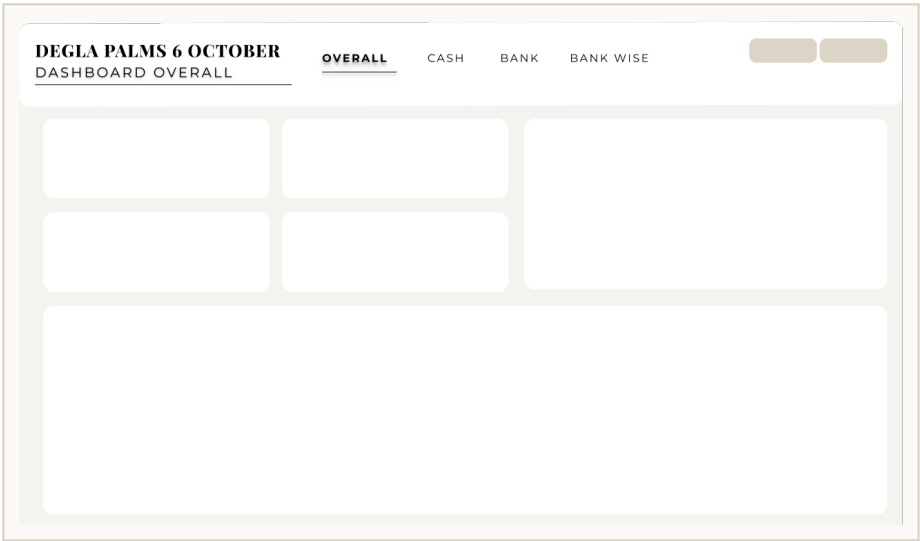


Figure 8.4 — Project Overall page template (Degla Palms).

8.5 Project Pages — Cash

Adds two **Deneb** custom-visual charts not available natively in Power BI: a Collection Rate curve and a Collected vs. Outstanding Amount curve, both plotted **by Installment Number** — this is what lets a viewer see exactly where in the payment schedule collection performance starts to fall off. The same KPI card set and detail pivot table from the Overall page repeat here, scoped to cash-channel payments.

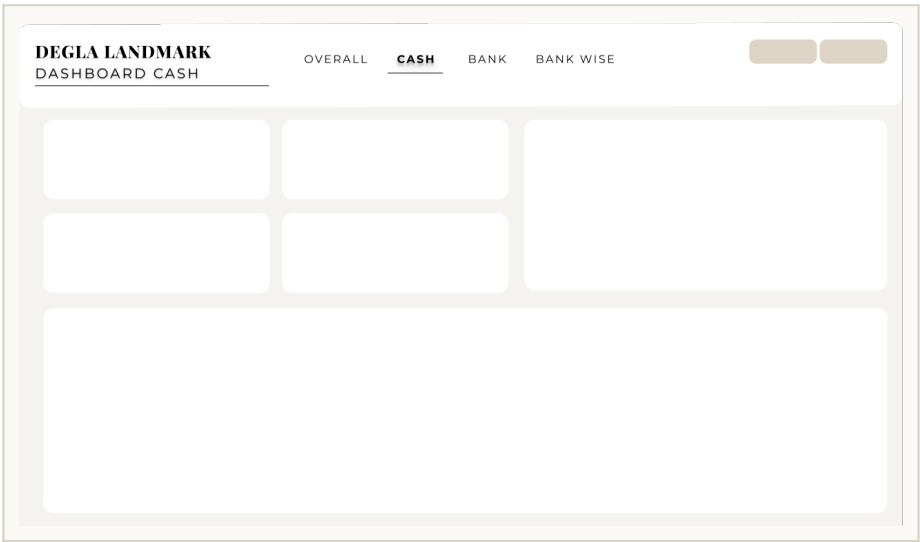


Figure 8.5 — Project Cash page template (Degla Landmark).

8.6 Project Pages — Bank

The same layout as the Cash page, with one addition: a **Bank slicer** (the Arabic-labelled Bank field) lets the user isolate a single receiving bank. The Collection Rate Deneb chart repeats here so bank-channel collection speed can be compared directly against the cash channel on the previous page.

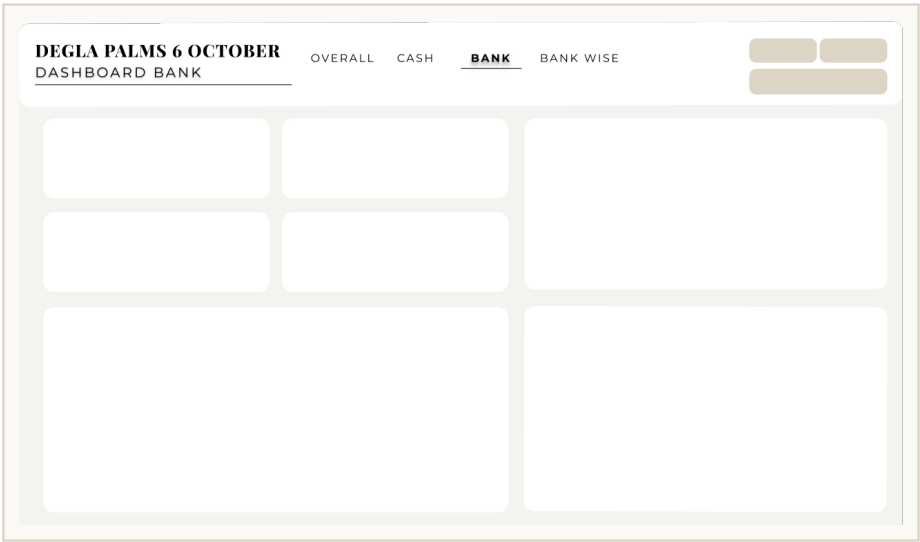


Figure 8.6 — Project Bank page template (Degla Palms).

8.7 Project Pages — Bank-wise

Ranks performance by individual bank rather than the bank channel as a whole: two Deneb charts (Collected Amount and Outstanding Amount, both by Bank) sit alongside a pivot table of Bank, Collected Amount, Outstanding Amount, Total Invoice Amount, Unit Price and a customer-row count — answering which specific banks settle reliably and which lag.

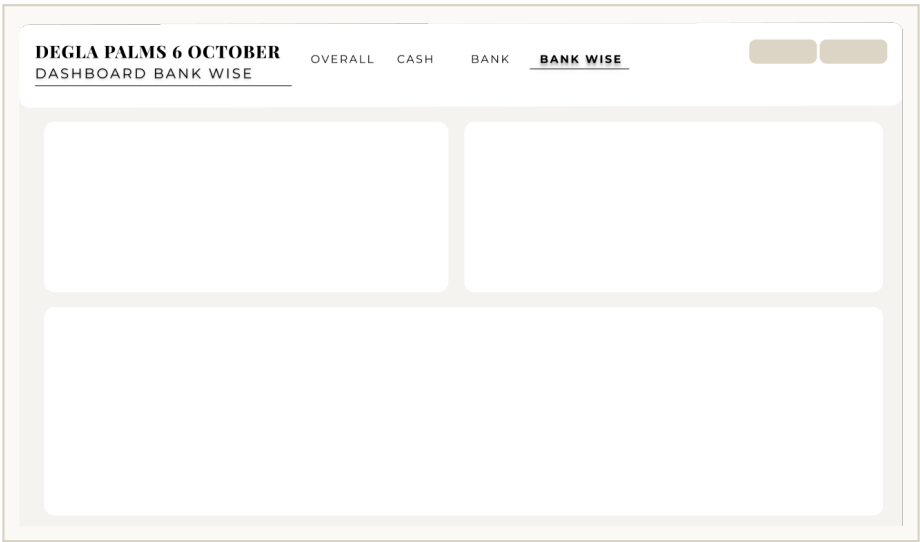


Figure 8.7 — Project Bank-wise page template (Degla Palms).

8.8 The Remaining Seven Projects

Degla Landmark, Crystal Plaza Maadi, Lake, Skyline Katameya, Rihana, Zahra and One Katameya each carry the identical four-page Overall / Cash / Bank / Bank-wise set described above, confirmed directly in the report's page list (§7.2) — 32 project pages in total, plus the 3 company-level pages, for 35 pages overall.

9 Interactivity

9.1 Slicers & Filters

Slicer / Filter	Field	Where it Appears
Date	Date (shared Date table)	Every Overall/Cash/Bank page, and the Collection by Date Summary
Bank	Bank (Arabic-labelled, Payment Dim)	Bank pages, for isolating one receiving bank
Project	Projects.Project	Row axis of the Global Summary and Collection by Date pivot tables
Payment Method	Payment Method (Payment Dim)	Legend/breakdown dimension on the Overall page's donut chart

9.2 Navigation & Buttons

All 35 pages carry **action buttons** for navigation — the Landing page alone has 11. Every project page repeats the same pair: a page-navigation action switching between Overall / Cash / Bank / Bank-wise, and a Home/Back action returning to the Landing page. The report's definition does not surface any explicit bookmark objects, so all navigation in this model runs on action buttons rather than bookmarks.

9.3 Cross Filtering

Because every dimension table relates to its project's fact table (§5.3), a single click or slicer change cascades automatically: **Filter** → **Visual** → **Result**. For example, moving the Date slicer on a Cash page filters the fact table by date, which flows through the relationship into every visual built on that fact table at once — the KPI cards recalculate, the donut chart redraws with only that period's installment states, and the pivot table's rows update — with no extra DAX required per visual, because the filter propagates through the model's relationships rather than through visual-level logic.

9.4 Deneb & Custom Visuals

Three custom visuals are registered in the report: **Deneb**, an **HTML content** visual, and a **pbigenie_hbc_solution** visual.

- **Deneb** — a Vega-Lite powered custom visual, used for the Collection Rate curve and the Collected vs. Outstanding Amount curves on every project's Cash, Bank and Bank-wise pages. It was chosen because native Power BI visuals cannot easily plot a smooth collection curve against installment number with the specific styling used across this report — Deneb's Vega-Lite grammar gives full control over that chart type.
- **HTML Content** — used on the Landing page, most likely for the branded title block seen in the background template (§7.1).
- **pbigenie_hbc_solution** — registered as a public custom visual in the report definition. It was not confirmed on any of the pages sampled for this documentation, so its specific use is noted here for completeness rather than described in detail.

10 Analysis & Insights

10.1 Sales Analysis

The sales layer answers what has been sold, at what value, and how far each project has progressed, using the measures defined in §6.3:

Measure	What it Shows
Adopted Total Units	Total inventory approved for sale in the project — the fixed denominator for Sold %.
Sold Count	Units actually contracted; the numerator of sales progress.
Project Completion %	Sold units as a share of adopted units, tracked as a construction-progress gauge.
Unit Price	Average contracted price, used to compare projects fairly against one another.
Total Invoice Amount	Full contracted value scheduled across all installments.
Customer Concentration	Collected value by customer, revealing the largest accounts in the portfolio.

10.2 Collection Analysis

The Collection layer (§6.4) answers how many invoices were raised and how many have actually been settled, and where the money still due is concentrated:

Measure	What it Shows
Issued Invoices	How many installments have become due for collection.
Collected Amount	The money actually in — what has been settled to date.
Outstanding Amount	The balance still due, at any date or installment number.
Collection Rate	The share of invoiced value recovered — the headline health metric per project.

10.3 Payment Analysis

Every installment is classified by payment method, so the same measure set reads consistently whether the viewer is looking at the whole portfolio, the cash channel only, or a single bank:

- **Paid / Due – Not Paid / Not Due** — the three Installment State values that drive every collection measure and the donut chart on each Overall page.
- **Cash** — walk-in and direct payments, tracked on each project's Cash page.
- **Bank** — payments settled through a bank, further split **Bank-wise** to rank individual banks by collected vs. outstanding value.

10.4 Key Business Insights

1. Collection rate is the real health signal — Sales progress and cash progress diverge. A project can be almost fully sold while its collection rate lags, so completion % alone overstates performance.

- 2. Outstanding balance concentrates in later installments** — Collected value falls as the installment number rises, which is where arrears accumulate and where follow-up effort pays back most.
- 3. Channel mix changes the risk profile** — Splitting cash from bank shows outstanding balance is not spread evenly between channels, and bank-wise ranking exposes which partners settle reliably.
- 4. A small set of customers drives the balance** — Collected amount by customer is heavily skewed, so a short priority list of accounts covers a large share of the outstanding value.
- 5. Comparability was the missing ingredient** — Identical measures across all eight projects turned isolated files into a ranking, making weak and strong portfolios obvious at a glance.

11 Conclusion

- Eight separate project files now run on one governed Power BI model.
- Collections, arrears and sales progress are visible in the same place, at any date.
- Cash and bank performance can finally be measured and compared per bank.
- Manual Excel reporting is replaced by a refreshable, auditable source of truth.

Across 35 report pages and roughly 51 tables in the combined star schemas, the project delivers what the original business problem (§2) was missing: one place to see what has been sold, invoiced, collected, and what remains outstanding — for the company as a whole, for any one of the eight projects, and for any payment channel or bank within it.

12 Future Scope

Automated Refresh & Alerts — Scheduled refresh with data alerts when overdue balance crosses a threshold.

Ageing Buckets & Risk Scoring — Classify arrears by days overdue and score customers by payment behaviour.

Forecasting — Project expected monthly collections from the remaining installment schedule.

Row-Level Security & Mobile — Per-project access for sales managers, plus a mobile layout for the field team.

Prepared by: Ahmed Ibrahim, Abdelaziz Ramy, Amina Elsayed, Nada Mohamed

El-Morshedy Real Estate — Data Analysis & Business Intelligence Graduation Project